

Steven Meyer

CS670 Artificial Intelligence

Prof. Arashdeep Kaur

8/2/2026

Cluedo: AI Exploration Edition

Project 2 Part 2 – Technical Documentation

Abstract

This project extends a graph-based command-line Cluedo game with a fully autonomous rational agent. The final system integrates:

- Configurable human, AI, and inactive player slots
- Private clockwise refutation
- Constraint-based knowledge representation
- Fixed-point logical inference
- Dijkstra shortest-path movement
- Strategic suggestions
- Formal accusations
- Elimination
- Persistent diagnostic logs
- Complete termination

Each AI models every card as a variable whose possible owners form a shrinking domain. Evidence from private hands, missed refutations, successful refutations, hand sizes, and envelope category rules drives general deductions without hard-coded scenario chains. Eight deterministic tests pass, and a seeded three-AI match terminated correctly on turn 38.

1. Introduction

The academic purpose of the project is to connect a complete game engine with core artificial-intelligence concepts: knowledge representation, logical deduction, constraint satisfaction, graph search, rational decision making, and reasoning under partial observability. Cluedo is a useful environment because every action changes both physical state and information state. Players must navigate a graph while inferring a hidden three-card solution from incomplete and private evidence.

1.1 Development Context

Part 1 established the reusable foundation: nine graph nodes representing rooms, weighted passages, four secret passages, six suspects, six weapons, randomized solution generation, even card dealing, clockwise turns, movement validation, mandatory room-based suggestions, and token teleportation. Part 2 retains those tested components and adds the intelligence and termination layers needed for a complete game.

1.2 Design Goals

- Model uncertainty explicitly instead of guessing.
- Produce advanced deductions from general constraints rather than scenario-specific conditionals.
- Keep private information private during standard play while making reasoning auditable during evaluation.
- Separate cards, movement, player state, knowledge, and orchestration into maintainable modules.
- Validate invalid inputs and edge cases without crashing or corrupting state.

1.3 Module Architecture

Module	Responsibility
<code>cards.py</code>	Cards, solution, deck, shuffle, and deal
<code>mansion.py</code>	Weighted graph, legal adjacent routes, and Dijkstra distances
<code>player.py</code>	Human/AI state, hands, notes, and elimination
<code>knowledge.py</code>	CSP domains, evidence, propagation, and solution detection
<code>game.py</code>	Setup, turns, refutation, decisions, accusations, and logging
<code>config.py/main.py</code>	Global DEBUG MODE and entry point

2. Implemented Game Rules

The program follows the classic project scope and excludes bonus cards, intrigue cards, and character-specific special abilities.

2.1 Setup and Turn Order

- Six fixed suspect slots are configured as Human, AI, or Inactive.
- Miss Scarlett is mandatory and begins the fixed clockwise order.
- Three to six active players and at least one AI are required.
- One suspect, one weapon, and one room are sealed in the confidential envelope.
- The other 18 cards are shuffled and distributed as evenly as possible.

2.2 Movement and Suggestions

A six-sided die determines whether an adjacent weighted edge is affordable. Entering a room ends movement and immediately requires a suggestion whose room component is locked to the destination. Suggested suspect and weapon tokens teleport to that room. A player moved by another suggestion may suggest immediately on the next turn or roll and leave. Secret passages are represented as zero-cost edges.

2.3 Refutation

The engine checks players in strict clockwise order beginning immediately left of the suggester. Each missed check is public evidence that the player holds none of the three cards. The first player with one or more matching cards reveals exactly one matching card privately to the suggester. The loop then stops. Eliminated players remain in this loop because their hands stay in play.

2.4 Accusation and Elimination

A formal accusation may be made at the beginning of a turn from any room. A correct accusation ends the game immediately. An incorrect accusation eliminates the player from future movement, suggestions, and accusations, but does not remove the hand. The game also terminates if only one eligible player remains.

3. AI Knowledge Representation

3.1 CSP Variable and Domain Model

The AI represents all 21 cards as variables. The initial domain of each card is the set of active players plus the Envelope. A card confirmed in the AI's own hand receives a singleton self-domain. The AI is removed from every other card domain. Every later observation either removes an impossible owner or confirms a single owner. This structure directly tracks:

- The AI's private cards
- Cards definitively possessed by opponents
- Cards opponents cannot possess
- The shrinking suspect, weapon, and room candidates for the confidential envelope

3.2 Refutation Constraints

When an opponent refutes a suggestion but the observing AI does not see the card, the system stores a disjunctive constraint: the refuter owns at least one card in the triplet. This preserves the uncertainty instead of selecting an arbitrary card. If later evidence removes the refuter from two card domains, the third becomes a required singleton.

3.3 Cardinality Constraints

- Each active player owns exactly the number of cards dealt to that player.
- The Envelope owns exactly one suspect, one weapon, and one room.
- If an owner has reached the known hand size, that owner is removed from all other domains.
- If an owner appears in exactly as many remaining domains as open hand slots, all those cards are confirmed.

3.4 Privacy Boundary

Each AI receives an independent KnowledgeBase object. Public passes and the identity of a refuter are shared with all agents. The actual shown card is provided only to the suggesting AI. Human players receive assistance logs but never another AI's private domain matrix during standard play.

4. Logical Deduction and Inference

4.1 Fixed-Point Propagation

After initialization and after every new observation, the engine enters a constraint-resolution loop. Each iteration evaluates disjunctions, player hand sizes, category envelope rules, singleton domains, and contradictions. If any domain changes, another iteration begins. The loop stops only when an entire pass produces no change, which is the fixed point.

4.2 Direct Elimination

A missed clockwise refutation removes that player from the domains of all three suggested cards. A privately revealed card immediately collapses to the refuter. Once an envelope card is confirmed for a category, the Envelope is removed from every other card in that category.

4.3 Advanced Inference Example

Suppose Colonel Mustard refutes Professor Plum, Rope, and Kitchen. The AI already knows Colonel Mustard does not own Rope and does not own Kitchen. The disjunctive constraint still requires him to own at least one card in the triplet. Professor Plum is therefore the only remaining candidate, so the general propagation engine confirms that Colonel Mustard owns Professor Plum.

```
DEBUG_MODE ADVANCED INFERENCE
```

```
[Colonel Mustard] ELIMINATE Colonel Mustard from Rope: prior evidence  
[Colonel Mustard] ELIMINATE Colonel Mustard from Kitchen: prior evidence  
[Colonel Mustard] ADD DISJUNCTION: Colonel Mustard owns at least one of  
['Kitchen', 'Professor Plum', 'Rope']  
[Colonel Mustard] Constraint iteration 1  
[Colonel Mustard] CONFIRM Professor Plum -> Colonel Mustard:  
only remaining card in disjunction  
[Colonel Mustard] Constraint iteration 2  
[Colonel Mustard] PROPAGATION FIXED POINT after 2 iteration(s)
```

Figure 1. Two exclusions reduce the refutation disjunction to one confirmed owner.

4.4 Bluff-Safe Unrefuted Suggestions

An unrefuted opponent suggestion does not automatically place all three cards in the Envelope, because the suggester may include cards from their own hand. A card is confirmed in the Envelope only when every other player passed and the AI had already proven that the suggester does not own that specific card.

5. AI Decision-Making

5.1 Movement Search

mansion.py implements Dijkstra's algorithm over the weighted room graph. The AI identifies unresolved room candidates and chooses the target with the greatest remaining information value, using shortest-path distance as a tie-breaker. After rolling, it selects the legal adjacent destination that minimizes the remaining distance to that target. A directly useful secret passage is selected at zero cost.

5.2 Strategic Suggestions

The AI scores suspect and weapon cards by the size of their ownership domains. Larger domains represent greater unresolved uncertainty, so suggesting those cards maximizes potential information gain. Cards not already in the AI hand receive preference. If another suggestion moved the AI token, the agent can use the new room immediately rather than spending a turn leaving it.

5.3 Rational Accusations

The AI never makes random accusations. `knowledge.solution()` returns a triplet only when exactly one suspect, one weapon, and one room each have the singleton Envelope domain. The AI then accuses at the start of its turn. This rule makes every AI accusation a consequence of the knowledge base rather than a probability guess.

5.4 Mean Turn-to-Convergence

The engine records the total game turn on which each AI first reaches a definitive solution. At termination it reports the arithmetic mean across agents that converged. In the seeded baseline integration run, the reported mean was 35.00 turns.

6. Diagnostic Logging

`config.py` contains the global `DEBUG_MODE` boolean. False provides standard play and hides background inference. True streams every elimination, confirmation, disjunction, propagation iteration, fixed point, and decision to the console. Regardless of the switch, every AI writes a permanent character-specific text log in the source directory.

7. Thorough Testing

7.1 Automated Test Suite

The deterministic unittest suite covers deck exclusion, even dealing, Dijkstra route cost, low-roll rejection, private-hand domains, negative evidence from passes, the required advanced inference, bluff-safe unrefuted inference, incorrect accusation elimination, and correct accusation termination.

```
AUTOMATED TEST EXECUTION

test_advanced_single_card_refutation_inference ... ok
test_correct_and_incorrect_accusations ... ok
test_deck_contains_eighteen_cards ... ok
test_even_dealing ... ok
test_passes_eliminate_possible_owners ... ok
test_private_hand_constraints ... ok
test_shortest_path_and_low_roll ... ok
test_unrefuted_card_can_be_confirmed_in_envelope ... ok

Ran 8 tests in 0.007s
OK
```

Figure 2. All eight deterministic tests completed successfully.

7.2 Full Integration Run

A three-AI match used seed 17 and ran without interactive decisions. It exercised autonomous dice rolls, shortest-path movement, suggestions, clockwise passes and refutations, private reveals, repeated propagation, convergence, accusation, and termination. Colonel Mustard correctly identified Mrs. Peacock, the Revolver, and the Ballroom on turn 38.

```
SEEDED FULL-GAME COMPLETION

[TURN 38] Colonel Mustard [AI] - Start of Turn
[Colonel Mustard] DECISION: definitive solution
('Mrs. Peacock', 'Revolver', 'Ballroom'); accuse now
FORMAL ACCUSATION by Colonel Mustard:
('Mrs. Peacock', 'Revolver', 'Ballroom')
CORRECT. Colonel Mustard wins immediately!
Solution: ('Mrs. Peacock', 'Revolver', 'Ballroom')

GAME TERMINATED
Winner: Colonel Mustard
Mean Turn-to-Convergence: 35.00 total game turns
```

Figure 3. Colonel Mustard makes a correct definitive accusation on turn 38.

7.3 Movement Validation and Low-Roll Handling

The first scenario accepts a legal edge when its cost does not exceed the die roll. The second scenario demonstrates the low-roll validation path: Ballroom has no adjacent edge costing two or less, so the move is rejected and the token remains in place.

```
FIGURE 4 - VALID MOVEMENT AND LOW-ROLL REJECTION

[MOVEMENT VALIDATION TEST]
Current room: Study | Dice roll: 3
Valid destinations: ['Kitchen', 'Hall', 'Library']
Selected route: Study -> Kitchen (cost 0)
RESULT: Valid adjacent path accepted.

[LOW-ROLL ERROR TEST]
Current room: Ballroom | Dice roll: 2
Valid destinations: []
Low-roll validation: no adjacent route is affordable.
RESULT: Illegal movement rejected; location unchanged.
```

Figure 4. Valid movement is accepted, while an unaffordable move is rejected.

7.4 Clockwise Refutation Scenarios

The first scenario shows a strict clockwise pass followed by a private reveal from the first player able to refute. The second scenario shows every opponent passing, producing a no-card-shown outcome. These observations are supplied to each AI according to the privacy rules.

FIGURE 5 - CLOCKWISE REFUTATION SCENARIOS

```
[STEP 2]: CONFIDENTIAL ENVELOPE GENERATION
> Extracting 1 Random Suspect Card... [CONFIDENTIAL]
> Extracting 1 Random Weapon Card... [CONFIDENTIAL]
> Extracting 1 Random Room Card... [CONFIDENTIAL]
>> Murder Envelope locked and sealed.

[STEP 3]: CARD DEALING & ENGINE INITIALIZATION
Remaining 18 cards shuffled and dealt clockwise.
[AI BOOT] Miss Scarlett: private matrix initialized.
[AI BOOT] Colonel Mustard: private matrix initialized.
[AI BOOT] Mrs. White: private matrix initialized.
[CLOCKWISE REFUTATION: PASS THEN PRIVATE REVEAL]
--- Clockwise Refutation Loop Started ---
> Checking Colonel Mustard... PASSES.
> Checking Mrs. White... REFUTES.
Mrs. White privately showed the AI one card.
[Miss Scarlett] PRIVATE REVEAL: Mrs. White showed Ballroom
--- Clockwise Refutation Loop Terminated ---

[CLOCKWISE REFUTATION: NO CARD SHOWN]
--- Clockwise Refutation Loop Started ---
> Checking Colonel Mustard... PASSES.
> Checking Mrs. White... PASSES.
No player could refute the suggestion.
--- Clockwise Refutation Loop Terminated ---
```

Figure 5. Clockwise pass/refute and completely unrefuted suggestion scenarios.

7.5 Incorrect Accusation and Edge-Case Termination

The incorrect accusation immediately removes Miss Scarlett from future turns while retaining her in the player list for card refutation. The second scenario eliminates two players and verifies that the only remaining eligible player wins and the game terminates.

FIGURE 6 - ELIMINATION AND LAST-PLAYER EDGE CASE

```
[INCORRECT ACCUSATION TEST]
FORMAL ACCUSATION by Miss Scarlett: ('Mrs. White', 'Rope', 'Study')
INCORRECT. Miss Scarlett is eliminated from turns, but retains cards and must continue refuting.
Turn eligibility after accusation: False
Card-refutation membership retained: True

[LAST ELIGIBLE PLAYER EDGE CASE]
Only Mrs. White remains eligible to win. Game over.
Winner: Mrs. White
Game over: True
```

Figure 6. Incorrect-accusation elimination and last-eligible-player termination.

8. Challenges Faced

8.1 Preserving Partial Knowledge

The most important challenge was representing a successful refutation when the observing AI knows the refuter but not the card. Treating that event as three independent possibilities loses the requirement that at least one must be true. The `RefutationConstraint` class preserves this disjunction and allows later evidence to resolve it.

8.2 Recursive Deductions

One confirmation can fill a hand, which eliminates that owner elsewhere, which can isolate an envelope card, which can resolve another disjunction. A one-pass update would miss this chain. Fixed-point iteration ensures every consequence is applied before decision-making resumes.

8.3 Eliminated Player State

An incorrect accusation does not remove a player from the ownership model. The implementation separates turn eligibility from membership in the clockwise refutation list, allowing eliminated players to retain cards and continue private refutation correctly.

8.4 Human Privacy

A shared console creates a risk of leaking hands and shown cards. Pass-device prompts, human-specific assistance files, independent AI logs, and the `DEBUG_MODE` boundary reduce this risk. Evaluation mode is intentionally explicit because it exposes internal reasoning.

9. Stability and Reliability

- Numeric and named menu inputs are validated in loops.
- Illegal player configurations are rejected and re-entered.
- Low rolls cannot traverse unaffordable edges.
- Contradictory CSP updates raise an explicit error instead of silently corrupting domains.
- Seeded randomness makes integration scenarios reproducible.
- Only Python standard-library modules are required by the submitted source code.

10. Additional Technical Details

Item	Implementation
Language	Python 3.12.13; compatible with Python 3.10+
External libraries	None required for gameplay
Core patterns	Dataclasses, composition, dependency injection for input, CSP domains, fixed-point iteration
Search	Dijkstra shortest paths on weighted adjacency dictionaries
Testing	unittest with deterministic seeds and temporary log directories
Persistence	UTF-8 text logs updated during runtime

10.1 Source-Code Quality

The implementation separates stable data definitions from stateful orchestration. Type hints and docstrings describe public responsibilities. Helper methods centralize input validation, menu resolution, AI access, and file output. The constraint engine is reusable independently of the command-line game, which allows direct unit testing of difficult inference cases.

10.2 Limitations

The interface is intentionally command-line based because the rubric prioritizes mechanics and AI over GUI presentation. The decision heuristic maximizes domain uncertainty but does not calculate full expected information gain over every possible refutation outcome. These choices keep the implementation explainable, testable, and within the classic-rule project scope.

11. Conclusion

The completed project combines a graph-search game engine with a general constraint-based reasoning system. It satisfies the required knowledge representation, direct and advanced inference, strategic movement and suggestions, private clockwise refutation, accusation and elimination, diagnostic logging, human assistance, testing, and termination mechanisms. Most importantly, its advanced deductions arise from reusable domain and cardinality constraints rather than hard-coded if-else chains.